

ECE/CS 552 Introduction to Computer Architecture
Fall 2023
University of Wisconsin-Madison
Prof. Joshua San Miguel

Midterm Exam
Friday, October 20, 2023
75 Minutes, In-Class

1. The exam is two-sided and has 14 pages, which includes one blank page, one page of blank pipeline diagrams and the two-page MIPS Green Card in the end.
2. Closed book/closed notes examination.
3. No handheld computers or portable computers allowed.
4. Calculator allowed.
5. A single 8.5"x11" double-sided, hand-written cheat sheet is allowed.
6. When applicable, show your work. Any partial points will be awarded based only on what appears in your answers.
7. No one should leave the room during the last 5 minutes of the examination.
8. Upon announcement of the end of the exam, stop writing on the exam paper immediately. Wait for proctors to pick up all exams. The instructor will announce when you may leave the room.
9. Failure to follow instructions may result in forfeiture of your exam and will be handled according to UWS 14 academic misconduct procedures.

FIRST (GIVEN) NAME: _____

LAST (FAMILY) NAME: _____ **SOLUTION** _____

STUDENT ID#: _____

	Achieved Marks	Maximum Marks
Problem 1		/ 6
Problem 2		/ 4
Problem 3		/ 6
Problem 4		/ 6
Problem 5		/ 10
Problem 6		/ 8
Problem 7		/ 4
Total		/ 44

Problem 1 (6 marks)

Consider the following instruction in our **MIPS** ISA from class:

```
addi $s0, $s1, 7
```

- (a) (4 marks) Provide the **32-bit machine encoding** of this instruction (in binary).

001000 10001 10000 0000000000000111

- (b) (2 marks) Using **only one instruction**, write MIPS assembly code that sets register \$t7 to 1 if the addi instruction above has **overflowed** and 0 otherwise. Assume that your code would run immediately after the addi instruction.

slt \$t7, \$s0, \$s1

Problem 2 (4 marks)

The following MIPS assembly code attempts to set the contents of register \$t0 to a number N that needs **more than 16 bits**. N[31:16] refers to an immediate value of the most-significant 16 bits of N, while N[15:0] refers to an immediate value of the least-significant 16 bits of N.

```
lui    $t0, <N[31:16]>
addi   $t0, $t0, <N[15:0]>
```

Give an **example number N** (in hexadecimal) where this code would **fail to set \$t0 correctly** and explain why it fails.

Any number where bit 15 is 1.

Example: 0x00008000, leads to [\$t0] = 0xFFFF8000.

The addi instruction sign-extends the immediate instead of zero-extending, which changes the most-significant bits of N that were put into \$t0 by the lui instruction.

Problem 3 (6 marks)

You write an encryption algorithm whose runtime consists of 50% arithmetic operations when running on a standard processor. You decide to apply new hardware optimizations to your processor in order to improve performance. Specifically, your optimizations simultaneously make **arithmetic operations 8x faster** and **memory operations 4x faster**. When running your encryption algorithm on this new optimized processor, you find that your algorithm is **twice** as fast as when running on the original standard processor.

What **fraction** of the original runtime (when running on the standard processor) is spent on **memory** operations? Show your work.

Let f be the fraction of time spent on memory operations in the standard processor.

Let g be the fraction of time spent on arithmetic operations in the standard processor.

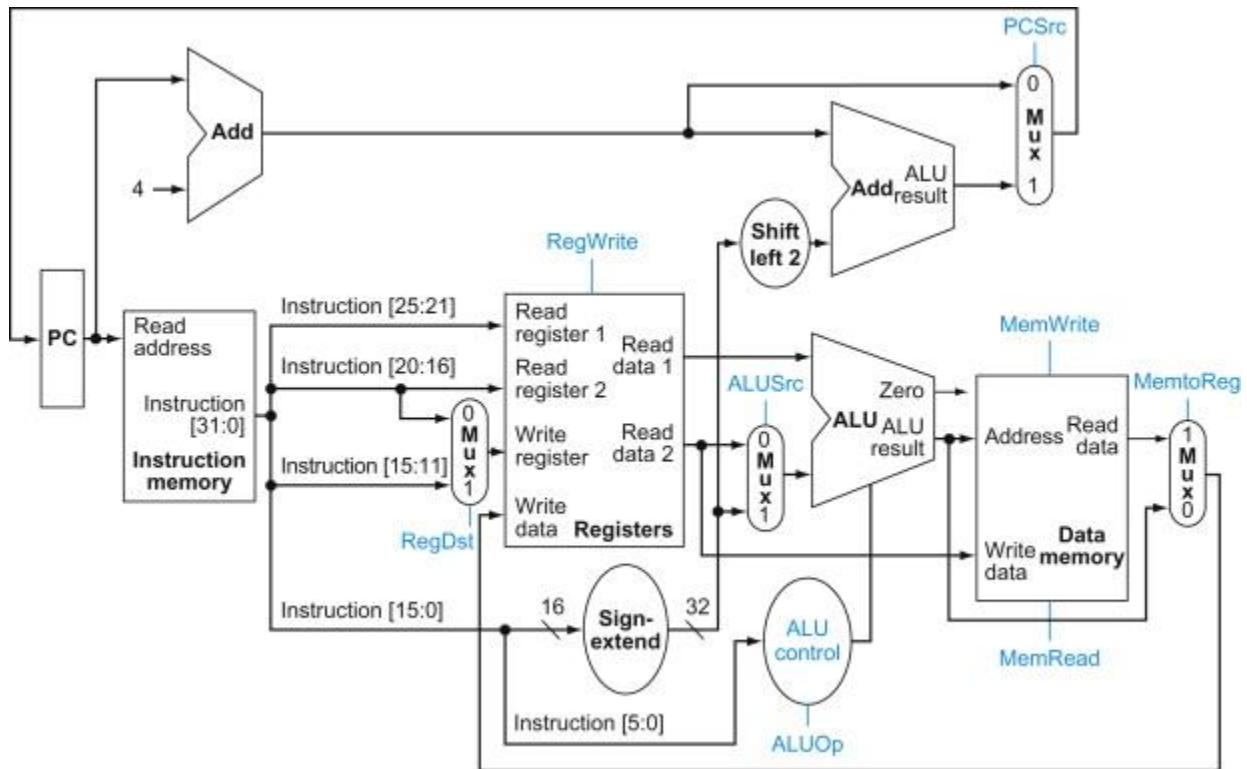
$$2 = \frac{1}{\frac{f}{4} + \frac{g}{8} + (1 - f - g)}$$

$$2 = \frac{1}{\frac{f}{4} + \frac{0.5}{8} + (0.5 - f)}$$

Thus $f = 1/12$.

Problem 4 (6 marks)

Consider our simple single-cycle processor from class, shown below (COD Figure 4.15).



You plan to add support for a new **R-format** MIPS instruction, **load-word-register**, that has the assembly syntax: **lwr rd, rt(rs)**, and performs the operation (using MIPS Green Card syntax):

$$R[rd] = M[R[rt] + R[rs]]$$

Supporting this instruction requires **no changes** to the processor data path. Fill in the tables below, specifying how each **control signal** must be set in order to perform this instruction.

Control Signal	Value (0, 1 or X)
RegWrite	1
RegDst	1
ALUSrc	0

Control Signal	Value (0, 1 or X)
MemWrite	0
MemRead	1
MemtoReg	1

Problem 5 (10 marks)

Consider the following MIPS assembly program:

```
L1: lw    $t0, 4($t2)
L2: bne   $t0, $zero, L1
L3: ori   $t2, $t1, 32
L4: sub   $t1, $s5, $s6
L5: sw    $t1, 0($s7)
```

- (a) (8 marks) List all the **register data dependences** (RAW, WAW, WAR). Specify the register and type of dependence as well as the older and younger instructions involved. One dependence is provided for you as an example.

\$t0, RAW, L1→L2

\$t2, WAR, L1→L3

\$t0, WAR, L2→L1

\$t1, WAR, L3→L4

\$t1, RAW, L4→L5

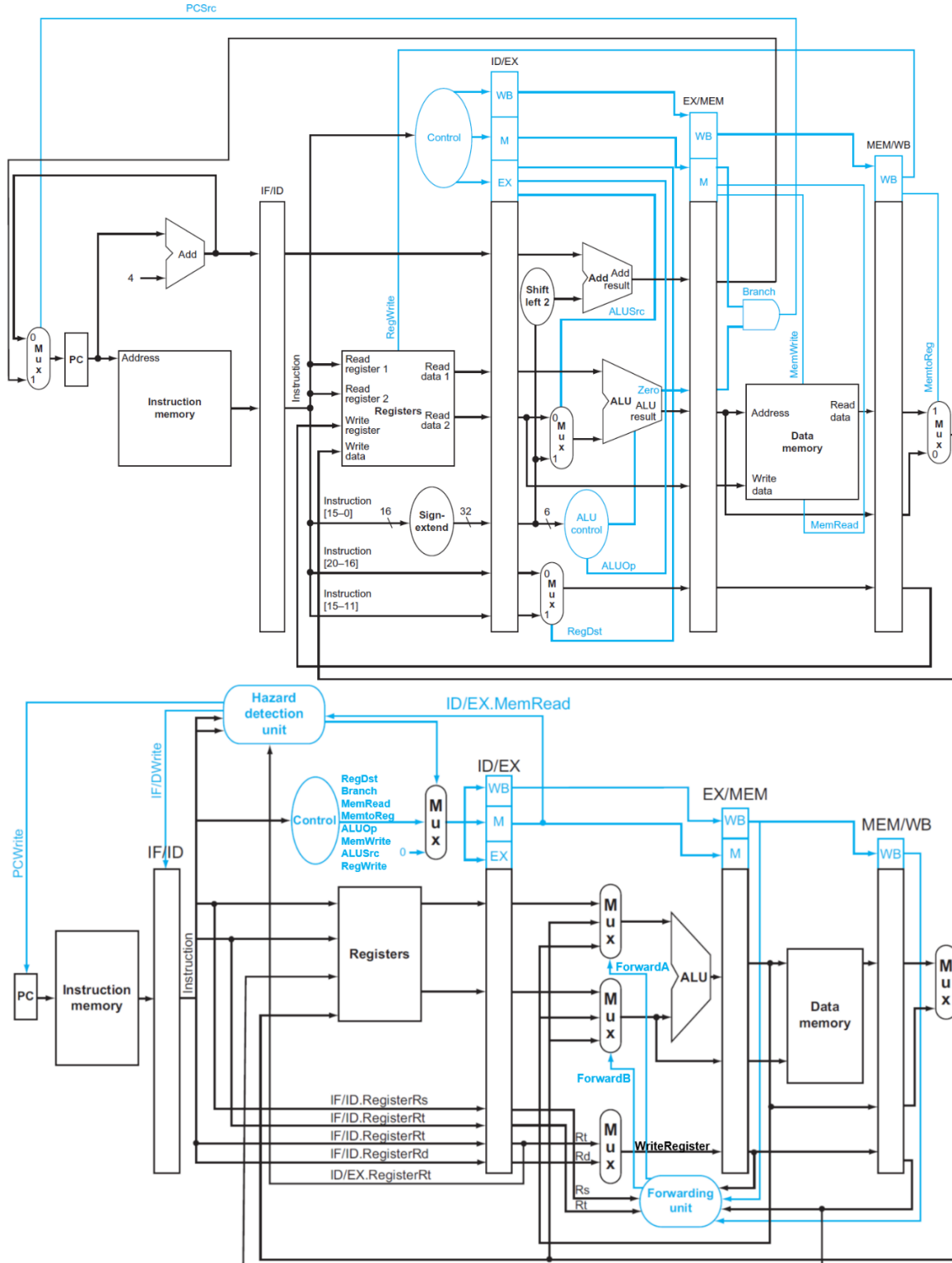
\$t0, WAW, L1→L1 (optional)

- (b) (2 marks) In no more than two sentences, explain why **WAR** register data dependences do **not** cause hazards in the five-stage pipelined processor from class.

Since 1) every instruction writes to its destination register in the same stage (WB), 2) every instruction reads from its source registers prior to the WB stage (ID), and 3) every instruction performs the read before the WB stage in the order specified by the program, it is not possible for a younger instruction in the program to write to a register before it is read by an older instruction.

Problem 6 (8 marks)

Consider our five-stage pipelined processor from class with support for **RF bypassing** and **forwarding to EX**, shown below. The top figure (COD Figure 4.51) shows the control signal details while the bottom figure (based on COD Figure 4.60) shows the forwarding details.



You run the following MIPS program on this processor. The **pipeline diagram** is shown below.

```
L1: lw $s0, 4($s1)
L2: add $s3, $s1, $s0
L3: sub $s2, $s0, $s3
L4: lw $s3, 8($s2)
L5: sw $s1, 4($s2)
```

	1	2	3	4	5	6	7	8	9	10
L1	F	D	X	M	W					
L2		F	D*	D	X	M	W			
L3			F*	F	D	X	M	W		
L4					F	D	X	M	W	
L5						F	D	X	M	W

- (a) (2 marks) Specify at which cycles (if any) the **MEM/WB.MemtoReg** control signal needs to be **equal to 1**.

Cycles 5, 9

- (b) (2 marks) Specify at which cycles (if any) the **ID/EX.RegWrite** control signal needs to be **equal to 1**.

Cycles 3, 5, 6, 7

- (c) (2 marks) Specify at which cycles (if any) **EX-to-EX** forwarding is **enabled**.

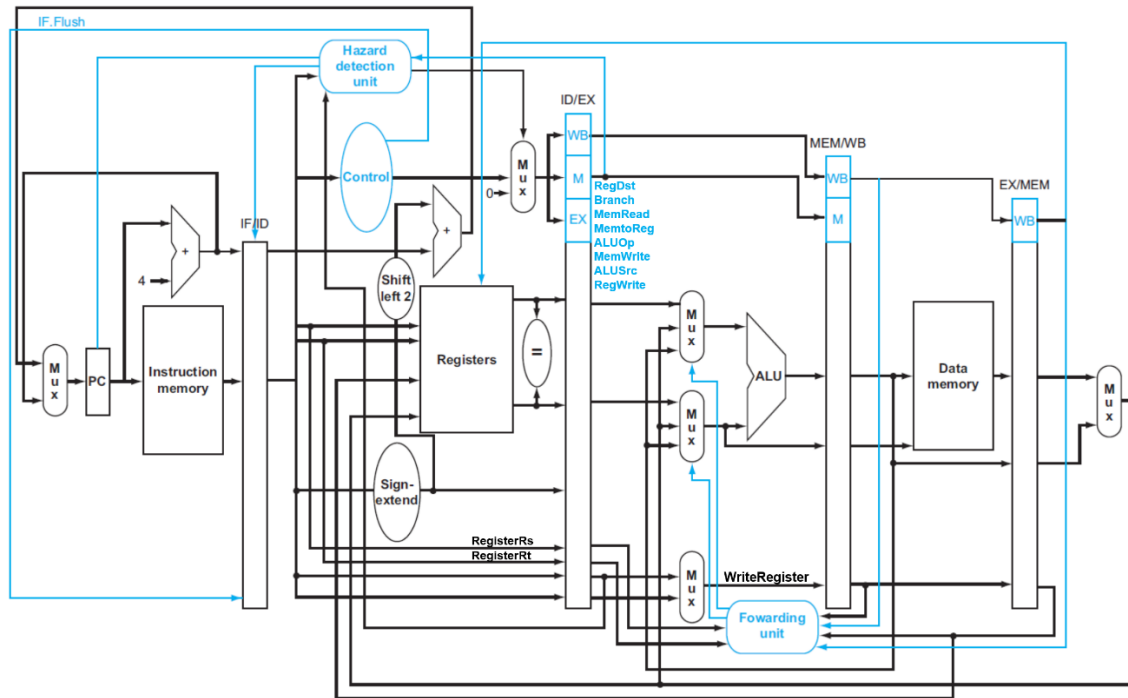
Cycles 6, 7

- (d) (2 marks) Specify at which cycles (if any) **MEM-to-EX** forwarding is **enabled**.

Cycles 5, 8

Problem 7 (4 marks)

Consider our five-stage pipelined processor from class that supports a **predict-not-taken** policy and **resolves branches in the ID stage**, shown below (based on COD Figure 4.65). Note that branch delay slots are not supported in this system.



You run the following program on this processor. Assume that **register \$s0** holds the value 0.

```

L1: beq $s0, $zero, L4
L2: lw  $t1, 0($t2)
L3: beq $s0, $zero, L5
L4: bne $s0, $zero, L3
L5: sub $t3, $t4, $t5

```

How many **cycles** does this program take to complete (starting from the IF of the first instruction to the WB of the last instruction)? You may **optionally** show your work in the pipeline diagrams on the next page.

**4 instructions are fetched (1 of which is flushed). There are no stall cycles.
Thus 8 cycles.**

This page provides blank pipeline diagrams, in case you need them.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

This page is left intentionally blank, in case you need extra space for answers.